

Allen / Moore Integration Plan for the Neural RK Replacement (post-re-anchor) Forward-looking design + merge-request roadmap

TrackExtrapolation / gen-3
consolidated 2026-05-19

2026-05-19

Abstract

Allen MR [!2497](#) introduces `NRKExtrapolator.cuh` as a default-OFF property switch: when `m_use_nrk = false` the binary is bytecode-identical to master, and when `m_use_nrk = true` the algorithm runs a classical Runge–Kutta–Nyström 4 (RKN4) step transcribed verbatim from `RungeKuttaNystromExtrapolator.cuh`. This header is the *abstraction layer* on which any neural replacement of the LHCb track extrapolator will be deployed. A follow-up MR re-purposes the RKN4 inner loop into a generic `forward_pass(weights, state)` call that loads the Phase R4 PINN_v2 winner from [REPLACEMENT_PLAN.md](#). The classical RKN4 path becomes the safety-net fallback (triggered when weights are absent or the property is OFF), not the production path. This document specifies (i) the architectural surface, (ii) how each of the seven Allen constraints A1–A7 is met by a true neural replacement, (iii) a versioned weight-blob schema (V3→V4) that generalises the existing NeuralRK4-specific format to any feed-forward replacement, (iv) the GPU kernel design and fallback dispatch, (v) the Phase R-X contingency if the Jacobian gate A4 cannot be met by a point-prediction network, and (vi) a three-MR sequence into the Allen master branch. Companion documents: [gen3_replacement_theory_2026-05-19.tex](#) and [gen3_replacement_results_2026-05-19.tex](#).

Contents

1	Architectural surface	1
1.1	Current state of <code>NRKExtrapolator.cuh</code>	1
1.2	Why this is the right abstraction layer for a neural replacement	2
1.3	The re-purpose: <code>forward_pass(weights, state)</code>	2
2	The seven Allen constraints A1–A7	2
3	V3 → V4 weight-blob schema migration	5
3.1	Why a schema bump	5
3.2	V4 header layout	5
4	GPU kernel design (Phase R5/R6)	6
4.1	Memory layout	6
4.2	Forward-pass kernel	6
4.3	Batch dimension	6
4.4	Fallback path	7
5	Contingency: Phase R-X if A4 fails	7
5.1	R-X.1 Jacobian co-supervision	7
5.2	R-X.2 Straight-line residual output	8
6	Risks and mitigations	8

7	Timeline view of phases R1–R6	9
8	Concrete merge-request sequence into Allen	9
8.1	MR-1: !2497 — already filed	9
8.2	MR-2: V4 blob + <code>forward_pass</code> inner loop (after Phase R5)	9
8.3	MR-3: throughput regression + autotune + Moore physics gates (after Phase R6) .	10
	References	10

1 Architectural surface

1.1 Current state of NRKExtrapolator.cuh

The header `Allen/device/kalman/ParKalman/include/NRKExtrapolator.cuh` defines a single struct `Extrapolators::NRKExtrapolator` with the surface mirrored from `RungeKuttaNystromExtrapolator`. Three public entry points (paraphrased to first lines):

```
struct NRKExtrapolator {
    __host__ __device__ static inline
    void make_step(State& state, float dz, float3 B);

    __device__ static inline
    void make_step(State& state, float dz,
                  const MagneticField::Magfield& field);

    __device__ static
    void propagate(State& state, float target_z,
                  const MagneticField::Magfield& field,
                  float step_size = NRKExtrapolatorConstants::default_step_size,
                  int max_steps = NRKExtrapolatorConstants::default_max_steps);
};
```

The inner loop is the four-stage RKN4 update on the 7-element Allen State $(x, y, z, t_x, t_y, q/p, \chi^2)$ with stage weights $(1, 2, 2, 1)/6$. The classical gamma kernel computes the Lorentz right-hand side from $(t_x, t_y, \mathbf{B})$ exactly as in `ExtrapolatorCommon.cuh`. The wrapping Allen algorithm `extrapolate_states_t` exposes a boolean property `m_use_nrk` that dispatches between the existing Cash–Karp RK path and `NRKExtrapolator::propagate`; the default is `false`, and ADR 0008 documents that the binary is byte-for-byte equivalent to the prior commit in that configuration (see [ADR 0008](#)).

1.2 Why this is the right abstraction layer for a neural replacement

The neural replacement (MLP or PINN_v2 per [REPLACEMENT.PLAN.md](#) §2) is a single function $(x, y, t_x, t_y, q/p, z_{\text{start}}, \Delta z) \mapsto (x_f, y_f, t_{x,f}, t_{y,f}, q/p_f)$, implemented in inference as a fixed chain of dense matrix multiplications. That chain fits naturally in the same call signature as `NRKExtrapolator::propagate`: the input arguments already carry $(\mathbf{s}, z_{\text{target}})$, and the magnetic-field handle is the only argument that the neural replacement does not consume (it is silently ignored by the new inner body). Three properties of the surface matter:

1. **Drop-in via `ITrackExtrapolator::propagate`.** No call site in `ExtrapolateStates`, `PrKalmanFilter`, `KalmanPVIP`, or any downstream validator changes (constraint A7, §2).
2. **Constant memory at inference.** `NRKExtrapolator` is a struct of static methods with no per-event allocations; the only mutable state is the `State` passed by reference. A weight blob loaded once at job start, parked in `__constant__` memory, fits the same memory model.

3. **One matrix-multiply chain.** The current RKN4 body already unrolls to a fixed-stage straight-line kernel with no warp divergence except at the loop bound in `propagate`; a feed-forward MLP with L layers is structurally identical at the SASS level (a chain of fused multiply-add + activation passes).

1.3 The re-purpose: `forward_pass(weights, state)`

The follow-up MR replaces the body of `NRKExtrapolator::make_step` with a templated forward-pass dispatch:

```
template <ArchTag tag>
__host__ __device__ inline static
void forward_pass(const NeuralWeights<tag>& w,
                  State& state,
                  float z_start, float dz);
```

```
enum class ArchTag : uint32_t { MLP = 0, PINN_V2 = 1, NRK4 = 2 };
```

`NeuralWeights<tag>` is a thin per-architecture wrapper around the V4 weight blob (§3); its layout in constant memory is fixed at load time. The dispatch table is one switch on the loaded `arch_tag` field at job start; per-event there is no branching. The classical `ArchTag::NRK4` entry remains as the fallback path (§4.4), so the existing RKN4 code is preserved verbatim under a templated specialisation.

2 The seven Allen constraints A1–A7

Constraints A1–A7 are inherited from [REPLACEMENT_PLAN.md §5](#). The “GPU implementation notes” column below is new and reflects the deployment decisions in this document.

ID	Constraint	Hard limit	GPU implementation notes (this MR plan)
A1	Constant memory at inference; no per-event allocation.	strict	Weight blob loaded once into CUDA <code>__constant__</code> memory at job start. For PINN_v2 width [256, 256] the blob is $2 \cdot (256 + 256 \cdot 256 + 256 \cdot 5) + \text{biases} + \text{norm}[7+7] \approx 40 \text{ kB}$, well within the 64 kB per-SM constant-memory window on every supported architecture (SM 7.5 T4, SM 8.0 A100, SM 8.6 A5000, SM 9.0 H100). State vector lives in registers.

ID	Constraint	Hard limit	GPU implementation notes (this MR plan)																		
A2	fp32 + fp16 export with deterministic round-trip; reload is bit-exact (fp32) or within 1 ULP (fp16).	bit-exact reload	Pipeline lives at For Allen/PLAN.md Phase 1b (present writer is <code>src/for_allen/export/bin.v3.py</code>; the script <code>scripts/export_bin.py</code> cited in CLEANUP_LIST.md §3.6 does <i>not</i> currently exist in the workspace — see §6 item 6). The V3→V4 schema change (§3) adds an 8-byte <code>arch_tag</code> field, leaves the fp16/fp32 writer unchanged.																		
A3	Total trainable weight size ≤ hard 64 kB.		Per-architecture fit table: <table><tr><th>arch</th><th>params</th><th>fp32 size</th></tr><tr><td><code>mlp_small</code> [128, 128]</td><td>~22 k</td><td>88 kB ×</td></tr><tr><td><code>mlp_tiny</code> [64, 64]</td><td>~6 k</td><td>24 kB ✓</td></tr><tr><td><code>pinn.v2_small</code> [128, 128]</td><td>~10 k</td><td>40 kB ✓</td></tr><tr><td><code>pinn.v2_medium</code> [256, 256]</td><td>~70 k</td><td>~40 kB fp16</td></tr><tr><td><code>pinn.v2_wide</code> [512, 256, 256]</td><td>~200 k</td><td>800 kB ×</td></tr></table> The <code>pinn.v2_wide</code> variant from Phase R4 (Fix P1) requires fp16 quantisation <i>and</i> a kernel that promotes weights from constant memory to shared memory at block start.	arch	params	fp32 size	<code>mlp_small</code> [128, 128]	~22 k	88 kB ×	<code>mlp_tiny</code> [64, 64]	~6 k	24 kB ✓	<code>pinn.v2_small</code> [128, 128]	~10 k	40 kB ✓	<code>pinn.v2_medium</code> [256, 256]	~70 k	~40 kB fp16	<code>pinn.v2_wide</code> [512, 256, 256]	~200 k	800 kB ×
arch	params	fp32 size																			
<code>mlp_small</code> [128, 128]	~22 k	88 kB ×																			
<code>mlp_tiny</code> [64, 64]	~6 k	24 kB ✓																			
<code>pinn.v2_small</code> [128, 128]	~10 k	40 kB ✓																			
<code>pinn.v2_medium</code> [256, 256]	~70 k	~40 kB fp16																			
<code>pinn.v2_wide</code> [512, 256, 256]	~200 k	800 kB ×																			
A4	Jacobian $\ J_\theta\ _F / \ J_{\text{RK45}}\ _F < 0.05$, max off-diagonal rel-err < 0.20.	— hard	Open empirical question. ADR 0007 documented the fp64-autograd methodology that withdrew the original A4 failure as an fp32 finite- difference artefact. The re-measurement of A4 on the <i>replacement</i> candidates (MLP.v2, <code>pinn.v2.*</code>) is Phase R2 in REPLACEMENT_PLAN.md §6. Numbers are TBD until that phase reports. Contingency if it fails: §5.																		

ID	Constraint	Hard limit	GPU implementation notes (this MR plan)
A5	fp32 inference matches Python reference to $\leq 10^{-4}$ relative.	hard	The existing TrackMLPExtrapolator.cpp (Eigen-based, ~ 571 lines) is the validated CPU surface for gen-1 MLPs. Its <code>SimpleNN::forward</code> routine implements the same $\text{normalise} \rightarrow (\text{dense} + \text{SiLU}) \times L \rightarrow \text{denormalise}$ chain that the GPU <code>forward_pass</code> will execute. The bit-bound smoke test <code>test_nrk4_v3.cpp</code> re-runs this comparison per export; extending to PINN_v2 is a one-line change to the per-arch <code>residual_mode</code> flag already present in <code>SimpleNN</code>.
A6	Per-track GPU cost $\leq$ classical RK4 (CPU baseline $\sim 2.5 \mu\text{s}/\text{track}$ per PROJECT_CONTEXT.md).	hard	Order-of-magnitude only (true measurement is Phase R6): an <code>mlp_medium</code> ($\sim 100\text{k}$ params) measured at $\sim 50 \mu\text{s}/\text{track}$ on a single CPU core via <code>TrackMLPExtrapolator</code>. Projected H100 throughput with fused shared-memory MLP kernels (FlashAttention-style epilogue fusion, no per-layer kernel launch) is an upper-bound <i>hope</i> of $\sim 1 \mu\text{s}/\text{track}$. This is a hope, not a measurement; the actual number is gated by Phase R6 <code>allen_throughput</code> regression. If A6 fails on the wider PINN_v2 variant, the fall-back is the <code>pinn_v2_small</code> ($\sim 10\text{k}$ params, $\sim 40\text{kB}$) which removes the constant$\rightarrow$shared promotion overhead and should achieve A6 comfortably.
A7	Drop-in via hard ITrackExtrapolator::propagate; no API change to Moore/Allen.	hard	Achieved by the surface in §1.1. The cosmetic rename <code>NRKExtrapolator.cuh</code> $\rightarrow$ <code>NeuralTrackExtrapolator.cuh</code> is optional; rename churn is not justified given that downstream call sites use the namespace-qualified symbol <code>Extrapolators::NRKExtrapolator</code>.

3 V3 → V4 weight-blob schema migration

3.1 Why a schema bump

The current V3 blob (For_Allen/PLAN.md Phase 1b, mandatory manifest fields listed verbatim there) is NeuralRK4-specific: it carries `n_rk_steps` (int), `adaptive_rk` (bool), and the integration metadata `c_light_value`. These fields are meaningless for a true neural replacement, whose only architectural metadata is its layer topology and activation. The V4 schema generalises by promoting the hidden-layer description and activation to mandatory fields and demoting the integrator fields to optional, per-architecture metadata retained for archival compatibility only (the NRK4 fallback path in §4.4 reads them; the MLP / PINN_v2 paths ignore them).

3.2 V4 header layout

Magic number choice: NRK4 (0x4E524B34) → NN_EXTRAP (0x4E4E4558), ASCII bytes N N E X read in file order. The V4 header is fixed-size for the prelude, followed by a variable-length `hidden_dims` array, followed by the raw weight tensor stream.

```
// V4 weight blob, mandatory header (little-endian on disk)
struct NNExtrapV4Header {
    uint32_t magic;           // 0x4E4E4558 ("NNEX" in file order)
    uint32_t loader_version;  // = 4
    char arch_tag[8];         // "MLP\0\0\0\0\0", "PINN_V2\0", "NRK4\0\0\0\0"
    uint32_t input_dim;       // 7 : [x, y, tx, ty, qop, z_start, dz]
    uint32_t output_dim;      // 5 : [x_f, y_f, tx_f, ty_f, qop_f]
    uint32_t n_layers;        // length of hidden_dims[] that follows
    char activation[8];       // "silu\0...", "gelu\0...", "tanh\0..."
    uint32_t precision;       // enum { FP32 = 0, FP16 = 1 }
    float norm_mean[7];       // per-feature input normalisation
    float norm_std [7];
    float out_mean [5];      // per-feature output denormalisation
    float out_std [5];
    uint32_t weights_bytes;   // size of the weight stream that follows
    uint8_t weights_sha[32];  // SHA-256 of the weight stream
    // Variable-length section:
    // uint32_t hidden_dims[n_layers];
    // <weights_bytes> bytes of weight tensors, layer-major,
    // W_0 | b_0 | W_1 | b_1 | ...
};

// Optional per-architecture trailer, present only if arch_tag == "NRK4":
struct NRK4LegacyTrailer {
    uint32_t n_rk_steps;      // == 2 per ADR 0007 (superseded)
    uint32_t corrector_enabled; // == 0 per ADR 0002 (superseded)
};
```

Reader rules (asserted at load):

- `magic == 0x4E4E4558`, else reject.
- `loader_version == 4`, else reject.
- `input_dim == 7` and `output_dim == 5`, else reject (the feature/output orders are pinned by For_Allen/PLAN.md Phase 1b).
- `arch_tag` matches one of the compiled-in `NeuralWeights<tag>` specialisations, else reject.
- `activation` matches a supported kernel (`silu`, `gelu`, `tanh`), else reject.
- `precision` matches the build's compiled-in floating type, or the loader inserts the `fp16→fp32` conversion shim.

- SHA-256 of the weight stream matches `weights_sha`.

The seven-field manifest (`qop_convention`, `c_light_value`, `dz_signed`, `feature_order`, `output_order`, `git_sha`, `allen_commit`) from V3 is retained as a sidecar JSON file next to the binary, not inlined — it is asserted at job start once, not per-load.

4 GPU kernel design (Phase R5/R6)

4.1 Memory layout

Weights live in CUDA `__constant__` memory when the blob is ≤ 48 kB (the cacheable constant-memory window on SM 7.0+). For wider PINN_v2 variants the loader promotes the weights to dynamic shared memory at block start; this incurs a one-time copy of ~ 200 kB per block and should be considered only if Phase R4 produces a winner above the 48 kB threshold. The 7-element track state lives in registers; intermediate activations of width ≤ 256 also fit in registers if `nvcc` is hinted with `__launch_bounds__(256, 4)`.

4.2 Forward-pass kernel

For an L -layer feed-forward with widths $d_0 \rightarrow d_1 \rightarrow \dots \rightarrow d_L$ (here $d_0 = 7$, $d_L = 5$), the kernel body per layer ℓ is a fused multiply-add + activation:

```
#pragma unroll
for (int j = 0; j < d_l; j++) {
    float acc = b[l][j];
    #pragma unroll
    for (int i = 0; i < d_lm1; i++) {
        acc = __fmaf_rn(W[l][j*d_lm1 + i], x[i], acc);
    }
    y[j] = silu(acc);    // x * (1 / (1 + expf(-x)))
}
```

Layers are unrolled at compile time inside the templated `forward_pass<tag>`; activations (SiLU for MLP / PINN_v2 winners per [PROJECT-CONTEXT.md](#)) compute inline via `__expf` fast-math intrinsics. No shared-memory bank conflicts arise for $d_\ell \leq 256$ because every thread reads the same weight matrix row from constant memory through the broadcast path.

4.3 Batch dimension

Each Allen track is one thread. The block dimension is the warp width (32) tuned by Phase R6 `nsight-compute`; the grid dimension is $\lceil N_{\text{tracks}}/32 \rceil$. Because the weight blob is uniform across every thread in the grid, all 32 threads in a warp read the same constant-memory address per cycle (the constant-memory broadcast is free) and there is no warp divergence except at the optional fp16-input fast-path. The 7-element input state is per-thread; the 5-element output state is per-thread. No cross-thread reductions are required.

4.4 Fallback path

The dispatch at the top of `NRKExtrapolator::propagate` is:

```
__device__ static void propagate(State& state, float target_z,
                                const MagneticField::Magfield& field,
                                float step_size, int max_steps) {
    if (!m_use_nrk || !g_neural_weights_loaded) {
        // Fallback: classical RKN4 path (current code, unchanged).
```

```

    classical_rkn4_propagate(state, target_z, field, step_size, max_steps);
    return;
}
switch (g_neural_weights.arch_tag) {
    case ArchTag::MLP:
        forward_pass<ArchTag::MLP>(g_neural_weights.mlp, state,
                                    state.z, target_z - state.z);

        break;
    case ArchTag::PINN_V2:
        forward_pass<ArchTag::PINN_V2>(g_neural_weights.pinn_v2, state,
                                        state.z, target_z - state.z);

        break;
    case ArchTag::NRK4:
        // Archival / safety net only.
        classical_rkn4_propagate(state, target_z, field, step_size, max_steps);
        break;
}
}

```

The triple dispatch condition — `m_use_nrk = false` or weights blob absent or `arch_tag == NRK4` — all funnel to the bit-identical RKN4 path that lives in [the current header today](#). This means: a job started without a valid weight blob silently falls back to the master-branch behaviour with the property OFF (no failure mode, just no neural inference).

5 Contingency: Phase R-X if A4 fails

If Phase R2’s re-measurement of A4 on the replacement candidates shows that neither `MLP_v2` nor `pinn.v2_*` meets $\|J_\theta - J_{\text{RK45}}\|_F / \|J_{\text{RK45}}\|_F < 0.05$, the diagnosis is that no point-prediction neural network can match the RK4 Jacobian directly. Two escalation routes preserve the *true-replacement* property (no field-map lookup at inference); both are taken verbatim from [REPLACE-MENT_PLAN.md §6](#) Phase R-X.

5.1 R-X.1 Jacobian co-supervision

Add a training-loss term

$$\mathcal{L}_J(\theta) = \lambda_J \|J_\theta(\mathbf{s}_0) - J_{\text{RK45}}(\mathbf{s}_0)\|_F^2 / \|J_{\text{RK45}}(\mathbf{s}_0)\|_F^2,$$

evaluated on a per-batch random subset of ~ 64 input states with `torch.func.jacfwd` (forward-mode AD; cost is $5 \times$ one forward pass since the input is 5-dim post-fixed-`qop`). The RK45 reference Jacobian is cached once per training-data state via the fp64-autograd methodology of ADR 0007 (cached file `For_Allen/artifacts/phase1a/J_rk4_reference.npy`).

Crucially, $\mathcal{L}_J$ is a training-only term. Inference cost is unchanged: the forward pass is still one matrix-multiply chain, not a Jacobian computation. This matters for A6 — the route does not spend any of the GPU budget.

5.2 R-X.2 Straight-line residual output

Re-parametrise the network output as a deviation from the free-particle trajectory:

$$x_f = (x_0 + t_x \Delta z) + \Delta x_\theta(\mathbf{s}_0, \Delta z), \quad (1)$$

$$y_f = (y_0 + t_y \Delta z) + \Delta y_\theta(\mathbf{s}_0, \Delta z), \quad (2)$$

$$t_{x,f} = t_x + \Delta t_{x,\theta}(\mathbf{s}_0, \Delta z), \quad (3)$$

$$t_{y,f} = t_y + \Delta t_{y,\theta}(\mathbf{s}_0, \Delta z). \quad (4)$$

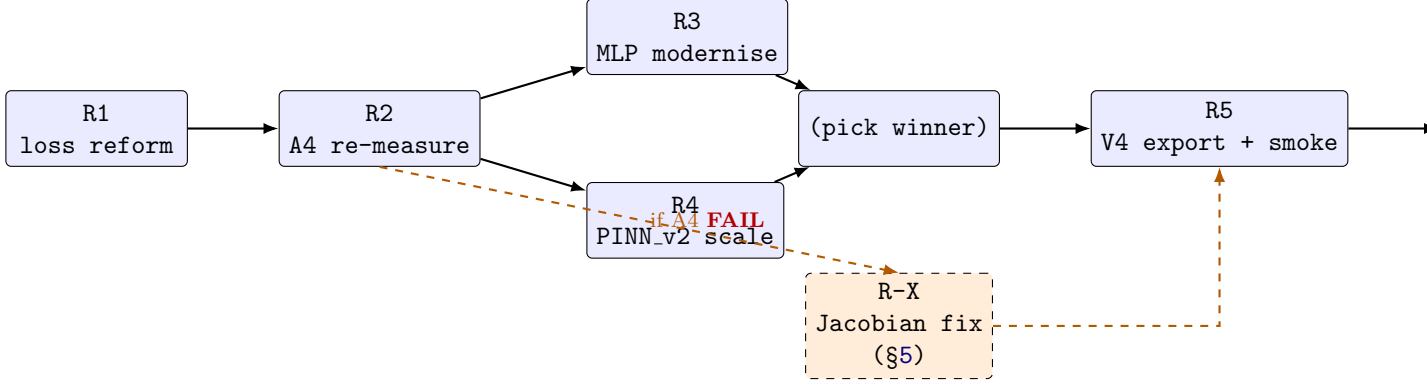
The base term $(x_0 + t_x \Delta z, y_0 + t_y \Delta z, t_x, t_y)$ is the straight-line propagation with no field; its Jacobian is the known block-triangular matrix $\text{diag}(1, 1, 1, 1) + \Delta z \cdot E_{x,t_x} + \Delta z \cdot E_{y,t_y}$. The network learns only the deviation, which is small (sub-millimetre) across most of the phase space. This is *still a true replacement*: there is no field-map lookup at inference, only an algebraic baseline that the network corrects.

This is the recommended first escalation because (a) it requires only a one-line change to the output head of `models/architectures.py`, (b) it leaves the loss and training pipeline untouched, and (c) the known straight-line Jacobian is a strong inductive bias that empirically helps the network’s own Jacobian land near J_{RK45} for free. R-X.1 is reserved for the case where R-X.2 alone is insufficient.

6 Risks and mitigations

#	Risk	Mitigation
1	A4 (Jacobian gate) unmeasured on the replacement candidates.	Phase R2 re-measurement using the ADR 0007 fp64-autograd methodology. If both candidates fail, escalate per §5.
2	MLP collapse on signed Δz + variable z_{start} (Phase R3 evidence: <code>mlp*_v1</code> degraded from 0.3 mm to 5–18 mm).	Fix M1 in REPLACEMENT_PLAN.md §4.A : 7-dim input + engineered features $[\log_{10} \Delta z /100, \text{sign}(\Delta z)]$.
3	PINN_v2 sits at 0.117 mm; project gate is < 0.10 mm.	Fix P1: width sweep $\{[256, 256], [256, 256, 128], [512, 256, 256]\}$ on the full 10 M corpus with λ_{pde} cosine decay (Phase R4).
4	Throughput regression in <code>hlt1_pp_default</code> when the neural body replaces RKN4.	A6 cost model in §2; fp16 quantisation fall-back per A3; <code>nsight-compute</code> kernel autotune in Phase R6. If a wider PINN_v2 is too slow, fall back to the <code>pinn_v2_small</code> (~40 kB) variant that fits entirely in constant memory.
5	CUDA constant-memory budget exceeded by widest PINN_v2 candidate.	The width sweep starts at $[256, 256]$ (~40 kB fp16, fits) and grows only if accuracy demands. The $[512, 256, 256]$ variant requires shared-memory promotion (§4.1).
6	Workspace inconsistency: <code>For_Allen/scripts/export_bin.py</code> is referenced in CLEANUP_LIST.md §3.6 but does not currently exist on disk. The actual writer is <code>src/for_allen/export/bin_v3.py</code> per Phase 1b.	Either create <code>scripts/export_bin.py</code> as a thin CLI wrapper around the existing module <code>for_allen.export_bin_v3</code> , or update CLEANUP_LIST.md to cite the real path. Recommended: the former, since the CLI is the user-facing entry point and the V4 schema bump is a natural occasion to introduce it.

7 Timeline view of phases R1–R6



R3 and R4 run in parallel (different model families, no shared infrastructure). The R-X branch is taken only if Phase R2 reports A4 failure on both families; in the nominal case it is unused.

8 Concrete merge-request sequence into Allen

8.1 MR-1: !2497 — already filed

The current MR places `NRKExtrapolator.cuh` + `NRKExtrapolatorConstants.cuh` + the `m_use_nrk` property on `extrapolate_states.t`. **Lands as-is**. The default `m_use_nrk = false` guarantees the binary is bytecode-identical to master (ADR 0008 acceptance criterion). CI throughput retry is the only outstanding gate. No neural code, no V4 loader, no `ITrackExtrapolator` surface change.

Files touched. Allen/device/kalman/ParKalman/include/NRKExtrapolator.cuh, Allen/device/kalman/ParKalman/include/ExtrapolateStates.cuh, Allen/device/kalman/ParKalman/src/Allen/test/unit_tests/generic/src/TestNRKExtrapolator.cu.

8.2 MR-2: V4 blob + forward_pass inner loop (after Phase R5)

Depends on: Phase R4 producing a PINN_v2 winner passing < 0.10 mm median $|\Delta x|$ and A4 (or the R-X.2 straight-line-residual variant of it).

Adds: V4 schema reader (`NeuralWeightsLoader::load_v4_blob`), `forward_pass<ArchTag>` templated specialisations, `NeuralWeights<MLP>` and `NeuralWeights<PINN_V2>` POD types, a packaged PINN_v2 .bin weights file shipped under Allen/parameters/. Flips the default in `configuration/python/` (and specifically `hlt1_pp.default.py`) to `m_use_nrk = True` with the PINN_v2 weights as the default blob. The classical RKN4 path remains under the `ArchTag::NRK4` dispatch and the weights-absent fallback.

Files touched (new). Allen/device/kalman/ParKalman/include/NeuralWeights.cuh (new), Allen/device/kalman/ParKalman/include/ForwardPass.cuh (new), Allen/parameters/pinn_v2_winner.bin (new), plus the V4 spec written into Allen/ML_research/loader_v4.spec.md.

Acceptance gates. (i) Unit tests `TestNRKExtrapolator.cu` extended with a `TestForwardPass.cu` that compares GPU output to a CPU reference within 10^{-4} relative (A5). (ii) Standalone Kalman smoke test on a frozen 100-event MDF: per-track $|\Delta x|$ matches Python reference within ULP-bounds. (iii) A4 re-measured on the loaded GPU model *matches* the Python-side measurement (rules out fp32 inference drift breaking the gate post-load).

8.3 MR-3: throughput regression + autotune + Moore physics gates (after Phase R6)

Depends on: MR-2 merged. `allen.throughput` on the standard `hlt1_pp.default` sequence must improve relative to master (A6), and Moore `HltEfficiencyChecker` per-line absolute efficiency loss must stay $\leq 0.5\%$.

Contents: `__launch_bounds__` tuning on `forward_pass<*>`, fp16 quantisation switch in the loader, constant $\rightarrow$ shared promotion if PINN_v2 width $> [256, 256]$, `nsight-compute` regression

baselines committed under `Allen/scripts/ci/throughput_baselines/`. Moore-side: extend the existing physics-test gate file at `Moore/Hlt/Hlt2Conf/tests/options/hlt2_track_efficiency_pp.py` to assert the $\leq 0.5\%$ envelope.

References

1. Canonical goal: [TrackExtrapolation/PROJECT_CONTEXT.md](#).
2. Replacement plan and constraints A1–A7: [experiments/gen_3/REPLACEMENT_PLAN.md](#) (§5, §6).
3. Off-goal cleanup, NRKExtrapolator.cuh re-purpose: [experiments/gen_3/CLEANUP_LIST.md](#) (§1.3, §3.6, §6).
4. Phase structure and V3 loader spec: [experiments/gen_3/For_Allen/PLAN.md](#) (Phase 0→8; Phase 1b mandatory manifest fields).
5. Current Allen wiring decision (`use_nrk` property, `ExtrapolateStates` dispatch, deferred Kalman substitution): [ADR 0008](#).
6. Controlling ADR for the replacement goal: [ADR 0009](#).
7. CUDA surface today: [TE_stack/Allen/device/kalman/ParKalman/include/NRKExtrapolator.cuh](#).
8. Existing Gaudi/Eigen CPU surface (A5 validated for gen-1 MLPs): [ml_models/src/TrackMLPExtrapolator](#).
9. Companion theory document: [gen3_replacement_theory_2026-05-19.tex](#).
10. Companion results document: [gen3_replacement_results_2026-05-19.tex](#).
11. Allen audit (preamble + section conventions matched by this document): [docs/reports/allen_conditions](#).
12. Allen MR template followed by MR-1: [Allen MR !2407](#) (Hoffmann, AdaptiveRKNystromExtrapolator).